Git Fundamentals - Day 1

Eng. Badr Aldien Soliman

1. What is Git and Why Use It?

What is Git?

Git is a **distributed version control system** that tracks changes in your code over time. Think of it as a time machine for your projects - you can see what changed, when it changed, and who changed it.

Why Use Git?

- **Track Changes**: See exactly what was modified in your code
- **Backup**: Never lose your work again
- **Collaboration**: Work with others without conflicts
- **Experimentation**: Try new features without breaking your main code
- **History**: Go back to any previous version of your project

Git History

- **Created by**: Linus Torvalds (also creator of Linux)
- **Released**: April 2005
- **Why created**: Needed a fast, distributed version control system for Linux kernel development after issues with existing tools

Important Note: Git ≠ GitHub

- **Git**: The version control system (runs on your computer)
- **GitHub**: A cloud platform that hosts Git repositories (we'll cover this tomorrow)

---

2. Installation and Setup

Installation

- **Windows**: Download from git-scm.com
- **Mac**: Install via Homebrew: `brew install git` or download from git-scm.com
- **Linux**: `sudo apt install git` (Ubuntu/Debian) or `sudo yum install git` (RHEL/CentOS)

Initial Configuration

After installation, set up your identity:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Verify your configuration:

```
git config --list
```

---

3. Understanding Repositories

What is a Repository (Repo)?

A repository is like a **special folder** that:

- Stores all versions of your code
- Tracks every change you make
- Contains the complete history of your project
- Can exist locally (on your computer) and remotely (on servers)

---

4. Creating Your First Repository

Step 1: Create a Project Folder

```
mkdir my-first-project
cd my-first-project
```

Step 2: Initialize Git

```
git init
```

This creates a hidden `.git` folder that stores all version control information.

Step 3: Change Default Branch Name

Modern practice uses `main` instead of `master`:

```
git branch -M main
```

---

5. Tracking Files and Making Commits

Check Repository Status

```
git status
```

This shows:

- **Untracked files**: New files Git doesn't know about
- **Modified files**: Changed files Git is watching
- **Staged files**: Files ready to be committed

Add Files to Tracking

```
# Add a specific file
git add filename.txt

# Add all files in current directory
git add .

# Add all files of a specific type
git add *.js
```

What is a Commit?

A **commit** is like taking a snapshot of your project at a specific moment. Each commit:

- Records what changed
- Includes a message describing the change
- Gets a unique ID (hash)
- Becomes part of your project's permanent history

Making Your First Commit

```
git commit -m "Add initial project files"
```

Commit Message Best Practices

Write commit messages in **imperative mood** (like giving commands):

✅ **Good Examples:**

- "Add user authentication"
- "Fix login bug"
- "Update navigation menu"
- "Remove unused CSS files"

❌ **Bad Examples:**

- "Added user authentication" (past tense)
- "Adding user authentication" (present continuous)
- "User auth" (too vague)
- "Fixed stuff" (not descriptive)

---

6. Viewing Project History

Check Commit History

```
git log
```

This shows:

- **Commit Hash**: Unique identifier (most important!)
- **Author**: Who made the commit
- **Date**: When it was made
- **Message**: Description of changes

Shorter Log Format

```
git log --oneline
```

---

7. Time Travel with Git

Go Back to Previous Commit

```
git checkout [commit-hash]
```

Example:

```
git checkout a1b2c3d
```

Understanding Detached HEAD

When you checkout a specific commit, you enter **"detached HEAD"** state:

- HEAD is like a pointer showing where you currently are
- Normally points to the latest commit of your current branch
- In detached HEAD, it points to a specific commit in history
- You can look around and make changes, but they won't be saved unless you create a new branch

Return to Latest Version

```
git checkout main
```

Force Return (Discard Changes)

```
git checkout -f main
```

⚠ **Warning**: This discards any uncommitted changes!

---

8. Working with Branches

What are Branches?

Branches let you work on different features simultaneously:

- **main**: Your stable, production-ready code
- **feature branches**: Experimental or new feature development

View All Branches

```
git branch
```

Create a New Branch

```
# Create and switch to new branch
git checkout -b new-feature

# Alternative: Create branch without switching
git branch new-feature
git checkout new-feature

# Copy from specific branch
git branch new-feature main
```

Branch Naming Conventions

Use **kebab-case** (lowercase with hyphens):

- ✅ `user-authentication`
- ✅ `fix-login-bug`
- ✅ `add-payment-system`
- ❌ `UserAuthentication` (PascalCase - not for branches)

- ❌ `user_authentication` (snake_case - not preferred)

Switch Between Branches

```
git checkout branch-name
```

Important: Branch Creation Timing

When you create a branch, it **copies the current state** of the code from wherever you are. So:

- Create from `main` to start fresh
- Create from another branch to build upon that work

---

9. File Management with Git

Deleting Files

Method 1: Delete from Computer Then Tell Git

```
rm filename.txt
git add filename.txt  # Stage the deletion
git commit -m "Remove filename.txt"
```

Method 2: Use Git to Delete

```
# Delete single file
git rm filename.txt

# Delete directory
git rm -r directory-name

git commit -m "Remove files"
```

Note: Avoid `rm -rf` as it's dangerous and can delete everything!

---

10. Typical Workflow Summary

1. **Start**: `git init` (once per project)
2. **Work**: Make changes to your files
3. **Check**: `git status` (see what changed)
4. **Stage**: `git add .` (prepare changes)
5. **Save**: `git commit -m "Descriptive message"`

6. **Repeat**: Steps 2-5 for each logical change
7. **Branch**: `git checkout -b feature-name` (for new features)
8. **History**: `git log` (review your progress)

---

11. Key Concepts Review

Essential Git Terms

- **Repository**: Project folder with version control
- **Commit**: Snapshot of your project at a point in time
- **Hash**: Unique identifier for each commit
- **Branch**: Parallel version of your code
- **HEAD**: Pointer to your current location
- **Staging Area**: Where files wait before being committed

Most Important Commands for Day 1

```
git init                    # Initialize repository
git status                  # Check current state
git add .                   # Stage all changes
git commit -m "message"     # Save changes
git log                     # View history
git checkout [hash/branch]  # Switch versions/branches
git branch                  # List branches
git checkout -b new-branch  # Create and switch to branch
```

Happy coding! badrsoliman.com